CSA0905-PROGRAMMING IN JAVA

CO5-AT2-APPLICATION BASED PROBLEM SOLVING

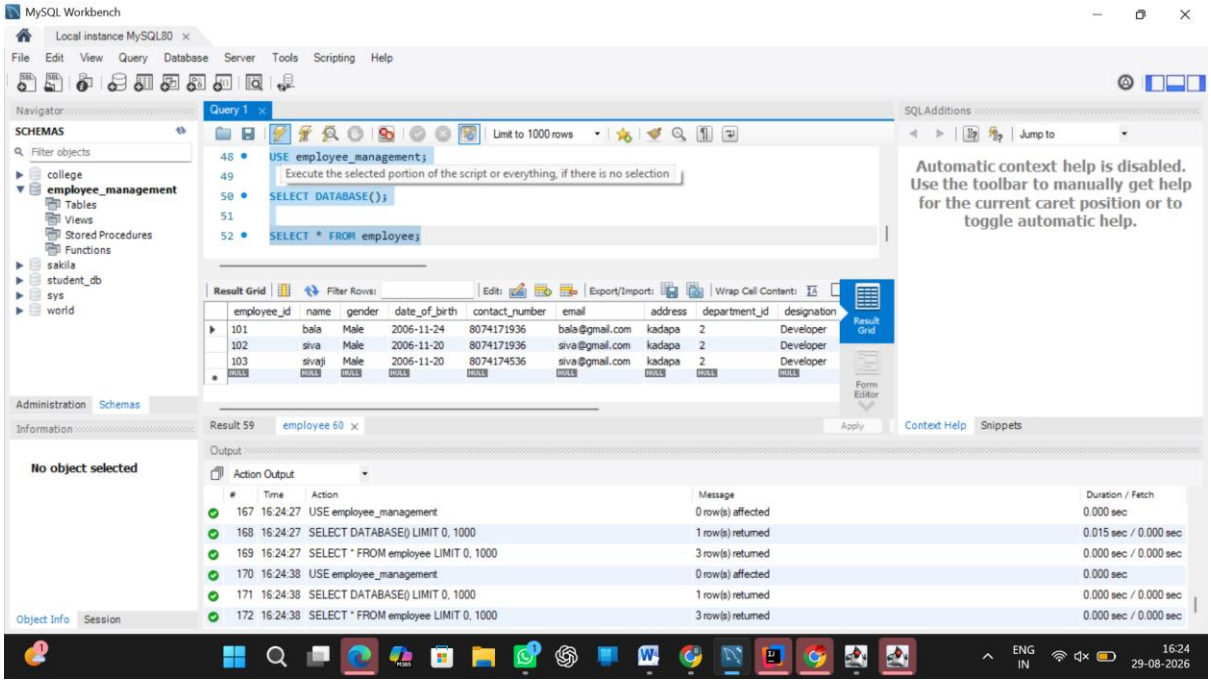
EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM USING JAVA AWT AND MYSQL

MYSQL CODE:

```
CREATE DATABASE IF NOT EXISTS employee_management;


USE employee_management;


DROP TABLE IF EXISTS employee;
DROP TABLE IF EXISTS department;


CREATE TABLE department (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL UNIQUE
);


INSERT INTO department VALUES
(1, 'HR'),
(2, 'IT'),
(3, 'Finance'),
(4, 'Marketing'),
(5, 'Sales');
```

```
CREATE TABLE employee (
    employee_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    gender VARCHAR(20) NOT NULL,
    date_of_birth VARCHAR(20) NOT NULL,
    contact_number VARCHAR(20) NOT NULL,
    email VARCHAR(100) NOT NULL,
    address VARCHAR(200) NOT NULL,
    department_id INT NOT NULL,
    designation VARCHAR(100) NOT NULL,
    date_of_joining VARCHAR(20) NOT NULL,
    basic_salary DOUBLE NOT NULL,
    allowances DOUBLE DEFAULT 0,
    deductions DOUBLE DEFAULT 0,
    employment_status VARCHAR(30) NOT NULL,

    CONSTRAINT fk_employee_department
    FOREIGN KEY (department_id)
    REFERENCES department(department_id)
);


SELECT * FROM department;

SELECT * FROM employee;

DESCRIBE employee;
```

OUTPUT:



JAVA CODES:

DBConnnection:

```
import java.awt.*;
import java.awt.event.*;

public class EmployeeManagement extends Frame implements ActionListener {

    Label l1, l2, l3, l4, l5, l6, l7, l8, l9, l10, l11, l12, l13, l14;
    TextField t1, t2, t3, t4, t5, t6, t7, t8, t9, t10, t11;
    Choice c1, c2;

    Button b1, b2, b3, b4, b5, b6, b7;

    TextArea ta;

    EmployeeDAO dao;

    public EmployeeManagement() {

        dao = new EmployeeDAO();

        setTitle("Employee Management and Payroll System");
        setSize(1000, 750);
        setLayout(new BorderLayout(10, 10));

        l1 = new Label("EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM",
Label.CENTER);
        add(l1, BorderLayout.NORTH);

        Panel formPanel = new Panel();
```

```
        formPanel.setLayout(new GridLayout(7, 4, 10, 10));

        l2 = new Label("Employee ID");
        t1 = new TextField();

        l3 = new Label("Name");
        t2 = new TextField();

        l4 = new Label("Gender");
        c1 = new Choice();
        c1.add("Male");
        c1.add("Female");

        l5 = new Label("Date of Birth");
        t3 = new TextField();

        l6 = new Label("Contact Number");
        t4 = new TextField();

        l7 = new Label("Email");
        t5 = new TextField();

        l8 = new Label("Address");
        t6 = new TextField();

        l9 = new Label("Department");
        c2 = new Choice();
        c2.add("HR");
        c2.add("IT");
        c2.add("Finance");
        c2.add("Marketing");
        c2.add("Sales");

        l10 = new Label("Designation");
        t7 = new TextField();

        l11 = new Label("Date of Joining");
        t8 = new TextField();

        l12 = new Label("Basic Salary");
        t9 = new TextField();

        l13 = new Label("Allowances");
        t10 = new TextField();

        l14 = new Label("Deductions");
        t11 = new TextField();

        formPanel.add(l2);
        formPanel.add(t1);

        formPanel.add(l3);
        formPanel.add(t2);

        formPanel.add(l4);
        formPanel.add(c1);

        formPanel.add(l5);
        formPanel.add(t3);

        formPanel.add(l6);
```

```
        formPanel.add(t4);

        formPanel.add(l7);
        formPanel.add(t5);

        formPanel.add(l8);
        formPanel.add(t6);

        formPanel.add(l9);
        formPanel.add(c2);

        formPanel.add(l10);
        formPanel.add(t7);

        formPanel.add(l11);
        formPanel.add(t8);

        formPanel.add(l12);
        formPanel.add(t9);

        formPanel.add(l13);
        formPanel.add(t10);

        formPanel.add(l14);
        formPanel.add(t11);

        add(formPanel, BorderLayout.CENTER);


        Panel bottomPanel = new Panel();
        bottomPanel.setLayout(new BorderLayout(5, 5));


        Panel buttonPanel = new Panel();
        buttonPanel.setLayout(new FlowLayout());

        b1 = new Button("ADD");
        b2 = new Button("SEARCH");
        b3 = new Button("UPDATE");
        b4 = new Button("DELETE");
        b5 = new Button("VIEW");
        b6 = new Button("PAYROLL");
        b7 = new Button("CLEAR");

        buttonPanel.add(b1);
        buttonPanel.add(b2);
        buttonPanel.add(b3);
        buttonPanel.add(b4);
        buttonPanel.add(b5);
        buttonPanel.add(b6);
        buttonPanel.add(b7);

        bottomPanel.add(buttonPanel, BorderLayout.NORTH);


        ta = new TextArea(8, 80);
        ta.setEditable(false);

        bottomPanel.add(ta, BorderLayout.CENTER);

        add(bottomPanel, BorderLayout.SOUTH);
```

```
        b1.addActionListener(this);
        b2.addActionListener(this);
        b3.addActionListener(this);
        b4.addActionListener(this);
        b5.addActionListener(this);
        b6.addActionListener(this);
        b7.addActionListener(this);


        addWindowListener(new WindowAdapter() {

            public void windowClosing(WindowEvent e) {
                dispose();
            }

        });

        setVisible(true);
    }


    public void actionPerformed(ActionEvent e) {

        try {

            if (e.getSource() == b1) {

                if (t1.getText().equals("") ||
                        t2.getText().equals("") ||
                        t3.getText().equals("") ||
                        t4.getText().equals("") ||
                        t5.getText().equals("") ||
                        t6.getText().equals("") ||
                        t7.getText().equals("") ||
                        t8.getText().equals("") ||
                        t9.getText().equals("")) {

                    ta.setText("Please enter all required details.");
                    return;
                }

                int id = Integer.parseInt(t1.getText());

                Employee old = dao.searchEmployee(id);

                if (old != null) {
                    ta.setText("Employee ID already exists.");
                    return;
                }

                Employee emp = getEmployeeFromFields();

                boolean result = dao.addEmployee(emp);

                if (result) {
                    ta.setText("Employee added successfully.");
                } else {
                    ta.setText("Failed to add employee.");
                }
```

```
            }


            if (e.getSource() == b2) {

                int id = Integer.parseInt(t1.getText());

                Employee emp = dao.searchEmployee(id);

                if (emp != null) {

                    t2.setText(emp.getName());
                    t3.setText(emp.getDateOfBirth());
                    t4.setText(emp.getContactNumber());
                    t5.setText(emp.getEmail());
                    t6.setText(emp.getAddress());
                    t7.setText(emp.getDesignation());
                    t8.setText(emp.getDateOfJoining());
                    t9.setText(String.valueOf(emp.getBasicSalary()));
                    t10.setText(String.valueOf(emp.getAllowances()));
                    t11.setText(String.valueOf(emp.getDeductions()));

                    ta.setText(
                            "Employee Found\n\n" +
                                    "Employee ID: " + emp.getEmployeeId() +
"\n" +
                                    "Name: " + emp.getName() + "\n" +
                                    "Gender: " + emp.getGender() + "\n" +
                                    "Department ID: " +
emp.getDepartmentId() + "\n" +
                                    "Designation: " + emp.getDesignation()
+ "\n" +
                                    "Basic Salary: " + emp.getBasicSalary()
+ "\n" +
                                    "Allowances: " + emp.getAllowances() +
"\n" +
                                    "Deductions: " + emp.getDeductions()
                    );

                } else {

                    ta.setText("Employee not found.");
                }
            }


            if (e.getSource() == b3) {

                int id = Integer.parseInt(t1.getText());

                Employee old = dao.searchEmployee(id);

                if (old == null) {
                    ta.setText("Employee not found.");
                    return;
                }

                Employee emp = getEmployeeFromFields();

                boolean result = dao.updateEmployee(emp);
```

```
                if (result) {
                    ta.setText("Employee updated successfully.");
                } else {
                    ta.setText("Failed to update employee.");
                }
            }


            if (e.getSource() == b4) {

                int id = Integer.parseInt(t1.getText());

                Employee old = dao.searchEmployee(id);

                if (old == null) {
                    ta.setText("Employee not found.");
                    return;
                }

                boolean result = dao.deleteEmployee(id);

                if (result) {
                    ta.setText("Employee deleted successfully.");
                    clearFields();
                } else {
                    ta.setText("Failed to delete employee.");
                }
            }


            if (e.getSource() == b5) {

                String result = dao.viewEmployees();

                if (result.equals("")) {
                    ta.setText("No employee records found.");
                } else {
                    ta.setText(result);
                }
            }


            if (e.getSource() == b6) {

                int id = Integer.parseInt(t1.getText());

                Employee emp = dao.searchEmployee(id);

                if (emp == null) {
                    ta.setText("Employee not found.");
                    return;
                }

                double grossSalary =
                        emp.getBasicSalary() + emp.getAllowances();

                double netSalary =
                        grossSalary - emp.getDeductions();

                ta.setText(
                        "PAYROLL DETAILS\n\n" +
```

```
                                "Employee ID: " + emp.getEmployeeId() +
"\n" +
                                "Name: " + emp.getName() + "\n\n" +
                                "Basic Salary: " + emp.getBasicSalary() +
"\n" +
                                "Allowances: " + emp.getAllowances() + "\n"
+
                                "Deductions: " + emp.getDeductions() +
"\n\n" +
                                "Gross Salary: " + grossSalary + "\n" +
                                "Net Salary: " + netSalary
                );
            }


            if (e.getSource() == b7) {

                clearFields();

                ta.setText("");
            }

        } catch (NumberFormatException ex) {

            ta.setText("Please enter valid numeric values.");

        } catch (Exception ex) {

            ta.setText("Error: " + ex.getMessage());
        }
    }


    public Employee getEmployeeFromFields() {

        Employee emp = new Employee();

        emp.setEmployeeId(Integer.parseInt(t1.getText()));
        emp.setName(t2.getText());
        emp.setGender(c1.getSelectedItem());
        emp.setDateOfBirth(t3.getText());
        emp.setContactNumber(t4.getText());
        emp.setEmail(t5.getText());
        emp.setAddress(t6.getText());

        int departmentId = 1;

        if (c2.getSelectedItem().equals("HR")) {
            departmentId = 1;
        } else if (c2.getSelectedItem().equals("IT")) {
            departmentId = 2;
        } else if (c2.getSelectedItem().equals("Finance")) {
            departmentId = 3;
        } else if (c2.getSelectedItem().equals("Marketing")) {
            departmentId = 4;
        } else if (c2.getSelectedItem().equals("Sales")) {
            departmentId = 5;
        }

        emp.setDepartmentId(departmentId);
```

```
        emp.setDesignation(t7.getText());
        emp.setDateOfJoining(t8.getText());

        emp.setBasicSalary(Double.parseDouble(t9.getText()));

        if (t10.getText().equals("")) {
            emp.setAllowances(0);
        } else {
            emp.setAllowances(Double.parseDouble(t10.getText()));
        }

        if (t11.getText().equals("")) {
            emp.setDeductions(0);
        } else {
            emp.setDeductions(Double.parseDouble(t11.getText()));
        }

        emp.setEmploymentStatus("Active");

        return emp;
    }


    public void clearFields() {

        t1.setText("");
        t2.setText("");
        t3.setText("");
        t4.setText("");
        t5.setText("");
        t6.setText("");
        t7.setText("");
        t8.setText("");
        t9.setText("");
        t10.setText("");
        t11.setText("");

        c1.select(0);
        c2.select(0);
    }
    public static void main(String[] args) {

        new EmployeeManagement();

    }
}
```

Employee:

```
public class Employee {

    private int employeeId;
    private String name;
    private String gender;
    private String dateOfBirth;
    private String contactNumber;
    private String email;
    private String address;
    private int departmentId;
    private String designation;
    private String dateOfJoining;
    private double basicSalary;
```

```
    private double allowances;
    private double deductions;
    private String employmentStatus;

    public int getEmployeeId() {
        return employeeId;
    }

    public void setEmployeeId(int employeeId) {
        this.employeeId = employeeId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getGender() {
        return gender;
    }

    public void setGender(String gender) {
        this.gender = gender;
    }

    public String getDateOfBirth() {
        return dateOfBirth;
    }

    public void setDateOfBirth(String dateOfBirth) {
        this.dateOfBirth = dateOfBirth;
    }

    public String getContactNumber() {
        return contactNumber;
    }

    public void setContactNumber(String contactNumber) {
        this.contactNumber = contactNumber;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    public int getDepartmentId() {
```

```
        return departmentId;
    }

    public void setDepartmentId(int departmentId) {
        this.departmentId = departmentId;
    }

    public String getDesignation() {
        return designation;
    }

    public void setDesignation(String designation) {
        this.designation = designation;
    }

    public String getDateOfJoining() {
        return dateOfJoining;
    }

    public void setDateOfJoining(String dateOfJoining) {
        this.dateOfJoining = dateOfJoining;
    }

    public double getBasicSalary() {
        return basicSalary;
    }

    public void setBasicSalary(double basicSalary) {
        this.basicSalary = basicSalary;
    }

    public double getAllowances() {
        return allowances;
    }

    public void setAllowances(double allowances) {
        this.allowances = allowances;
    }

    public double getDeductions() {
        return deductions;
    }

    public void setDeductions(double deductions) {
        this.deductions = deductions;
    }

    public String getEmploymentStatus() {
        return employmentStatus;
    }

    public void setEmploymentStatus(String employmentStatus) {
        this.employmentStatus = employmentStatus;
    }
}
```

EmployeeDAO:

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class EmployeeDAO {

    public boolean addEmployee(Employee e) {

        try {
            Connection con = DBConnection.getConnection();

            String sql = "INSERT INTO employee VALUES (?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?, ?, ?)";

            PreparedStatement ps = con.prepareStatement(sql);

            ps.setInt(1, e.getEmployeeId());
            ps.setString(2, e.getName());
            ps.setString(3, e.getGender());
            ps.setString(4, e.getDateOfBirth());
            ps.setString(5, e.getContactNumber());
            ps.setString(6, e.getEmail());
            ps.setString(7, e.getAddress());
            ps.setInt(8, e.getDepartmentId());
            ps.setString(9, e.getDesignation());
            ps.setString(10, e.getDateOfJoining());
            ps.setDouble(11, e.getBasicSalary());
            ps.setDouble(12, e.getAllowances());
            ps.setDouble(13, e.getDeductions());
            ps.setString(14, e.getEmploymentStatus());

            ps.executeUpdate();

            ps.close();
            con.close();

            return true;

        } catch (Exception ex) {
            System.out.println("Add Employee Error");
            ex.printStackTrace();
            return false;
        }
    }


    public Employee searchEmployee(int id) {

        Employee e = null;

        try {
            Connection con = DBConnection.getConnection();

            String sql = "SELECT * FROM employee WHERE employee_id=?";

            PreparedStatement ps = con.prepareStatement(sql);

            ps.setInt(1, id);
```

```
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {

                e = new Employee();

                e.setEmployeeId(rs.getInt("employee_id"));
                e.setName(rs.getString("name"));
                e.setGender(rs.getString("gender"));
                e.setDateOfBirth(rs.getString("date_of_birth"));
                e.setContactNumber(rs.getString("contact_number"));
                e.setEmail(rs.getString("email"));
                e.setAddress(rs.getString("address"));
                e.setDepartmentId(rs.getInt("department_id"));
                e.setDesignation(rs.getString("designation"));
                e.setDateOfJoining(rs.getString("date_of_joining"));
                e.setBasicSalary(rs.getDouble("basic_salary"));
                e.setAllowances(rs.getDouble("allowances"));
                e.setDeductions(rs.getDouble("deductions"));
                e.setEmploymentStatus(rs.getString("employment_status"));
            }

            rs.close();
            ps.close();
            con.close();

        } catch (Exception ex) {
            System.out.println("Search Employee Error");
            ex.printStackTrace();
        }

        return e;
    }


    public boolean updateEmployee(Employee e) {

        try {
            Connection con = DBConnection.getConnection();

            String sql = "UPDATE employee SET name=?, gender=?,
date_of_birth=?, contact_number=?, email=?, address=?, department_id=?,
designation=?, date_of_joining=?, basic_salary=?, allowances=?,
deductions=?, employment_status=? WHERE employee_id=?";

            PreparedStatement ps = con.prepareStatement(sql);

            ps.setString(1, e.getName());
            ps.setString(2, e.getGender());
            ps.setString(3, e.getDateOfBirth());
            ps.setString(4, e.getContactNumber());
            ps.setString(5, e.getEmail());
            ps.setString(6, e.getAddress());
            ps.setInt(7, e.getDepartmentId());
            ps.setString(8, e.getDesignation());
            ps.setString(9, e.getDateOfJoining());
            ps.setDouble(10, e.getBasicSalary());
            ps.setDouble(11, e.getAllowances());
            ps.setDouble(12, e.getDeductions());
            ps.setString(13, e.getEmploymentStatus());
            ps.setInt(14, e.getEmployeeId());
```

```
            int rows = ps.executeUpdate();

            ps.close();
            con.close();

            return rows > 0;

        } catch (Exception ex) {
            System.out.println("Update Employee Error");
            ex.printStackTrace();
            return false;
        }
    }


    public boolean deleteEmployee(int id) {

        try {
            Connection con = DBConnection.getConnection();

            String sql = "DELETE FROM employee WHERE employee_id=?";

            PreparedStatement ps = con.prepareStatement(sql);

            ps.setInt(1, id);

            int rows = ps.executeUpdate();

            ps.close();
            con.close();

            return rows > 0;

        } catch (Exception ex) {
            System.out.println("Delete Employee Error");
            ex.printStackTrace();
            return false;
        }
    }


    public String viewEmployees() {

        String result = "";

        try {
            Connection con = DBConnection.getConnection();

            String sql = "SELECT e.employee_id, e.name, d.department_name,
e.designation, e.basic_salary, e.allowances, e.deductions,
e.employment_status FROM employee e JOIN department d ON
e.department_id=d.department_id";

            PreparedStatement ps = con.prepareStatement(sql);

            ResultSet rs = ps.executeQuery();

            while (rs.next()) {

                result += "ID: " + rs.getInt("employee_id") + "\n";
```

```
                result += "Name: " + rs.getString("name") + "\n";
                result += "Department: " + rs.getString("department_name")
+ "\n";
                result += "Designation: " + rs.getString("designation") +
"\n";
                result += "Basic Salary: " + rs.getDouble("basic_salary") +
"\n";
                result += "Allowances: " + rs.getDouble("allowances") +
"\n";
                result += "Deductions: " + rs.getDouble("deductions") +
"\n";
                result += "Status: " + rs.getString("employment_status") +
"\n";
                result += "-----------------------------\n";
            }

            rs.close();
            ps.close();
            con.close();

        } catch (Exception ex) {
            System.out.println("View Employee Error");
            ex.printStackTrace();
        }

        return result;
    }
}
```

Employee Management:

```
import java.awt.*;
import java.awt.event.*;

public class EmployeeManagement extends Frame implements ActionListener {

    Label l1, l2, l3, l4, l5, l6, l7, l8, l9, l10, l11, l12, l13, l14;
    TextField t1, t2, t3, t4, t5, t6, t7, t8, t9, t10, t11;
    Choice c1, c2;

    Button b1, b2, b3, b4, b5, b6, b7;

    TextArea ta;

    EmployeeDAO dao;

    public EmployeeManagement() {

        dao = new EmployeeDAO();

        setTitle("Employee Management and Payroll System");
        setSize(1000, 750);
        setLayout(new BorderLayout(10, 10));

        l1 = new Label("EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM",
Label.CENTER);
        add(l1, BorderLayout.NORTH);

        Panel formPanel = new Panel();
```

```
        formPanel.setLayout(new GridLayout(7, 4, 10, 10));

        l2 = new Label("Employee ID");
        t1 = new TextField();

        l3 = new Label("Name");
        t2 = new TextField();

        l4 = new Label("Gender");
        c1 = new Choice();
        c1.add("Male");
        c1.add("Female");

        l5 = new Label("Date of Birth");
        t3 = new TextField();

        l6 = new Label("Contact Number");
        t4 = new TextField();

        l7 = new Label("Email");
        t5 = new TextField();

        l8 = new Label("Address");
        t6 = new TextField();

        l9 = new Label("Department");
        c2 = new Choice();
        c2.add("HR");
        c2.add("IT");
        c2.add("Finance");
        c2.add("Marketing");
        c2.add("Sales");

        l10 = new Label("Designation");
        t7 = new TextField();

        l11 = new Label("Date of Joining");
        t8 = new TextField();

        l12 = new Label("Basic Salary");
        t9 = new TextField();

        l13 = new Label("Allowances");
        t10 = new TextField();

        l14 = new Label("Deductions");
        t11 = new TextField();

        formPanel.add(l2);
        formPanel.add(t1);

        formPanel.add(l3);
        formPanel.add(t2);

        formPanel.add(l4);
        formPanel.add(c1);

        formPanel.add(l5);
        formPanel.add(t3);

        formPanel.add(l6);
```

```
        formPanel.add(t4);

        formPanel.add(l7);
        formPanel.add(t5);

        formPanel.add(l8);
        formPanel.add(t6);

        formPanel.add(l9);
        formPanel.add(c2);

        formPanel.add(l10);
        formPanel.add(t7);

        formPanel.add(l11);
        formPanel.add(t8);

        formPanel.add(l12);
        formPanel.add(t9);

        formPanel.add(l13);
        formPanel.add(t10);

        formPanel.add(l14);
        formPanel.add(t11);

        add(formPanel, BorderLayout.CENTER);


        Panel bottomPanel = new Panel();
        bottomPanel.setLayout(new BorderLayout(5, 5));


        Panel buttonPanel = new Panel();
        buttonPanel.setLayout(new FlowLayout());

        b1 = new Button("ADD");
        b2 = new Button("SEARCH");
        b3 = new Button("UPDATE");
        b4 = new Button("DELETE");
        b5 = new Button("VIEW");
        b6 = new Button("PAYROLL");
        b7 = new Button("CLEAR");

        buttonPanel.add(b1);
        buttonPanel.add(b2);
        buttonPanel.add(b3);
        buttonPanel.add(b4);
        buttonPanel.add(b5);
        buttonPanel.add(b6);
        buttonPanel.add(b7);

        bottomPanel.add(buttonPanel, BorderLayout.NORTH);


        ta = new TextArea(8, 80);
        ta.setEditable(false);

        bottomPanel.add(ta, BorderLayout.CENTER);

        add(bottomPanel, BorderLayout.SOUTH);
```

```
        b1.addActionListener(this);
        b2.addActionListener(this);
        b3.addActionListener(this);
        b4.addActionListener(this);
        b5.addActionListener(this);
        b6.addActionListener(this);
        b7.addActionListener(this);


        addWindowListener(new WindowAdapter() {

            public void windowClosing(WindowEvent e) {
                dispose();
            }

        });

        setVisible(true);
    }


    public void actionPerformed(ActionEvent e) {

        try {

            if (e.getSource() == b1) {

                if (t1.getText().equals("") ||
                        t2.getText().equals("") ||
                        t3.getText().equals("") ||
                        t4.getText().equals("") ||
                        t5.getText().equals("") ||
                        t6.getText().equals("") ||
                        t7.getText().equals("") ||
                        t8.getText().equals("") ||
                        t9.getText().equals("")) {

                    ta.setText("Please enter all required details.");
                    return;
                }

                int id = Integer.parseInt(t1.getText());

                Employee old = dao.searchEmployee(id);

                if (old != null) {
                    ta.setText("Employee ID already exists.");
                    return;
                }

                Employee emp = getEmployeeFromFields();

                boolean result = dao.addEmployee(emp);

                if (result) {
                    ta.setText("Employee added successfully.");
                } else {
                    ta.setText("Failed to add employee.");
                }
```

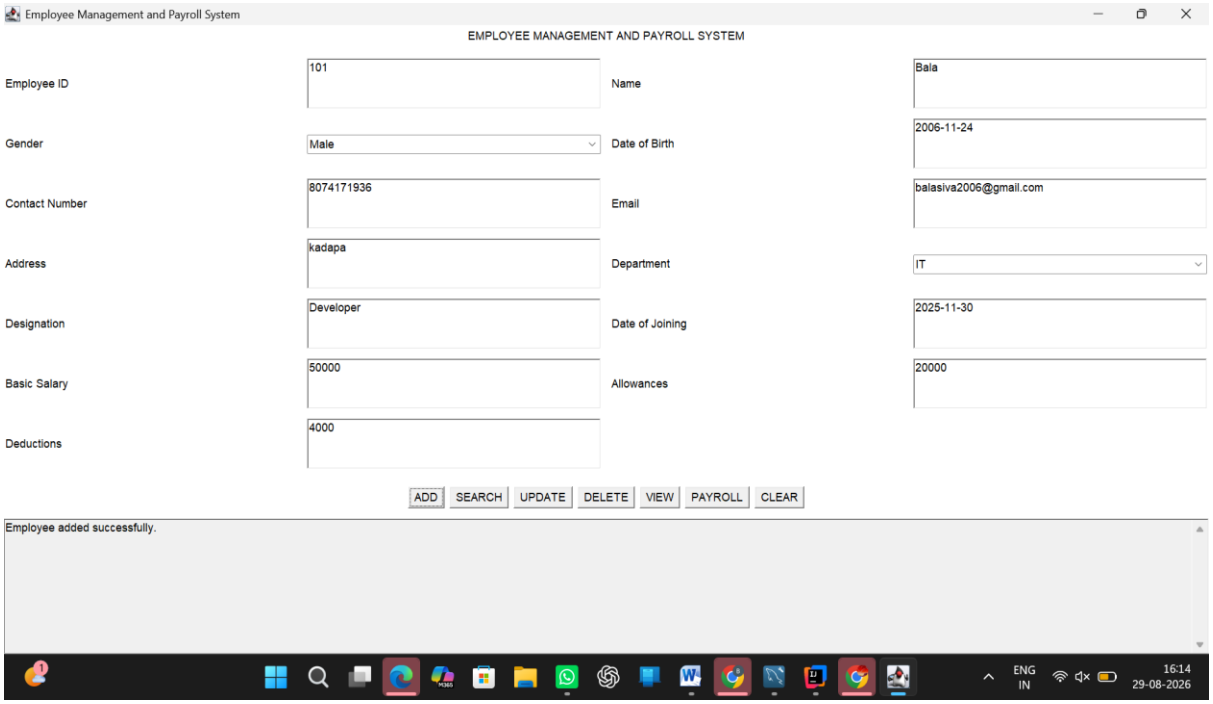
```
            }


            if (e.getSource() == b2) {

                int id = Integer.parseInt(t1.getText());

                Employee emp = dao.searchEmployee(id);

                if (emp != null) {

                    t2.setText(emp.getName());
                    t3.setText(emp.getDateOfBirth());
                    t4.setText(emp.getContactNumber());
                    t5.setText(emp.getEmail());
                    t6.setText(emp.getAddress());
                    t7.setText(emp.getDesignation());
                    t8.setText(emp.getDateOfJoining());
                    t9.setText(String.valueOf(emp.getBasicSalary()));
                    t10.setText(String.valueOf(emp.getAllowances()));
                    t11.setText(String.valueOf(emp.getDeductions()));

                    ta.setText(
                            "Employee Found\n\n" +
                                    "Employee ID: " + emp.getEmployeeId() +
"\n" +
                                    "Name: " + emp.getName() + "\n" +
                                    "Gender: " + emp.getGender() + "\n" +
                                    "Department ID: " +
emp.getDepartmentId() + "\n" +
                                    "Designation: " + emp.getDesignation()
+ "\n" +
                                    "Basic Salary: " + emp.getBasicSalary()
+ "\n" +
                                    "Allowances: " + emp.getAllowances() +
"\n" +
                                    "Deductions: " + emp.getDeductions()
                    );

                } else {

                    ta.setText("Employee not found.");
                }
            }


            if (e.getSource() == b3) {

                int id = Integer.parseInt(t1.getText());

                Employee old = dao.searchEmployee(id);

                if (old == null) {
                    ta.setText("Employee not found.");
                    return;
                }

                Employee emp = getEmployeeFromFields();

                boolean result = dao.updateEmployee(emp);
```

```
            if (result) {
                ta.setText("Employee updated successfully.");
            } else {
                ta.setText("Failed to update employee.");
            }
        }


        if (e.getSource() == b4) {

            int id = Integer.parseInt(t1.getText());

            Employee old = dao.searchEmployee(id);

            if (old == null) {
                ta.setText("Employee not found.");
                return;
            }

            boolean result = dao.deleteEmployee(id);

            if (result) {
                ta.setText("Employee deleted successfully.");
                clearFields();
            } else {
                ta.setText("Failed to delete employee.");
            }
        }


        if (e.getSource() == b5) {

            String result = dao.viewEmployees();

            if (result.equals("")) {
                ta.setText("No employee records found.");
            } else {
                ta.setText(result);
            }
        }


        if (e.getSource() == b6) {

            int id = Integer.parseInt(t1.getText());

            Employee emp = dao.searchEmployee(id);

            if (emp == null) {
                ta.setText("Employee not found.");
                return;
            }

            double grossSalary =
                    emp.getBasicSalary() + emp.getAllowances();

            double netSalary =
                    grossSalary - emp.getDeductions();

            ta.setText(
                    "PAYROLL DETAILS\n\n" +
```

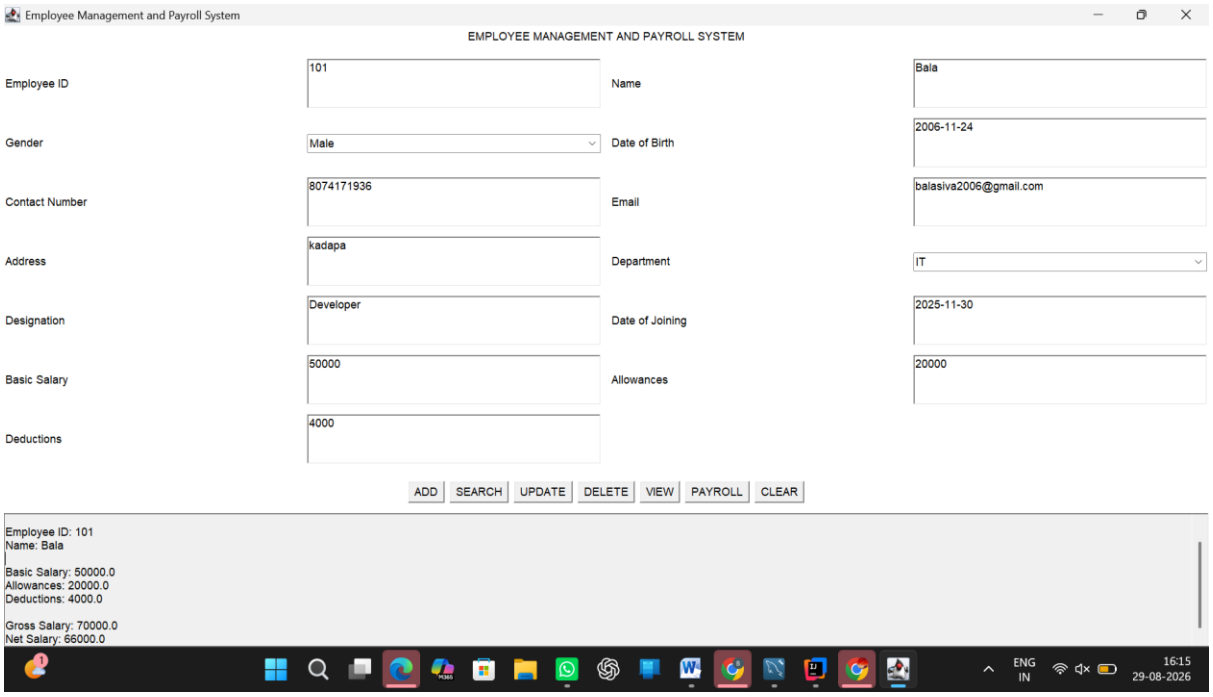
```
                                "Employee ID: " + emp.getEmployeeId() +
"\n" +
                                "Name: " + emp.getName() + "\n\n" +
                                "Basic Salary: " + emp.getBasicSalary() +
"\n" +
                                "Allowances: " + emp.getAllowances() + "\n"
+
                                "Deductions: " + emp.getDeductions() +
"\n\n" +
                                "Gross Salary: " + grossSalary + "\n" +
                                "Net Salary: " + netSalary
                );
            }


            if (e.getSource() == b7) {

                clearFields();

                ta.setText("");
            }

        } catch (NumberFormatException ex) {

            ta.setText("Please enter valid numeric values.");

        } catch (Exception ex) {

            ta.setText("Error: " + ex.getMessage());
        }
    }


    public Employee getEmployeeFromFields() {

        Employee emp = new Employee();

        emp.setEmployeeId(Integer.parseInt(t1.getText()));
        emp.setName(t2.getText());
        emp.setGender(c1.getSelectedItem());
        emp.setDateOfBirth(t3.getText());
        emp.setContactNumber(t4.getText());
        emp.setEmail(t5.getText());
        emp.setAddress(t6.getText());

        int departmentId = 1;

        if (c2.getSelectedItem().equals("HR")) {
            departmentId = 1;
        } else if (c2.getSelectedItem().equals("IT")) {
            departmentId = 2;
        } else if (c2.getSelectedItem().equals("Finance")) {
            departmentId = 3;
        } else if (c2.getSelectedItem().equals("Marketing")) {
            departmentId = 4;
        } else if (c2.getSelectedItem().equals("Sales")) {
            departmentId = 5;
        }

        emp.setDepartmentId(departmentId);
```

```
        emp.setDesignation(t7.getText());
        emp.setDateOfJoining(t8.getText());

        emp.setBasicSalary(Double.parseDouble(t9.getText()));

        if (t10.getText().equals("")) {
            emp.setAllowances(0);
        } else {
            emp.setAllowances(Double.parseDouble(t10.getText()));
        }

        if (t11.getText().equals("")) {
            emp.setDeductions(0);
        } else {
            emp.setDeductions(Double.parseDouble(t11.getText()));
        }

        emp.setEmploymentStatus("Active");

        return emp;
    }


    public void clearFields() {

        t1.setText("");
        t2.setText("");
        t3.setText("");
        t4.setText("");
        t5.setText("");
        t6.setText("");
        t7.setText("");
        t8.setText("");
        t9.setText("");
        t10.setText("");
        t11.setText("");

        c1.select(0);
        c2.select(0);
    }


    public static void main(String[] args) {

        new EmployeeManagement();

    }
}
```

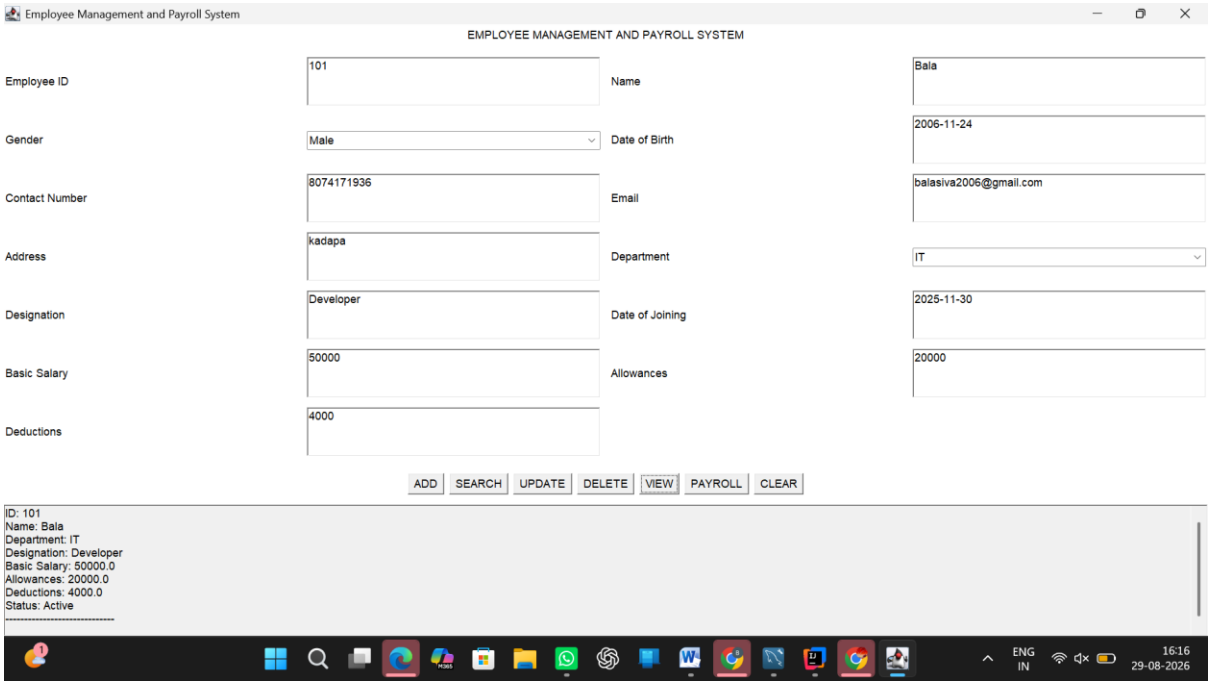
OUTPUT:



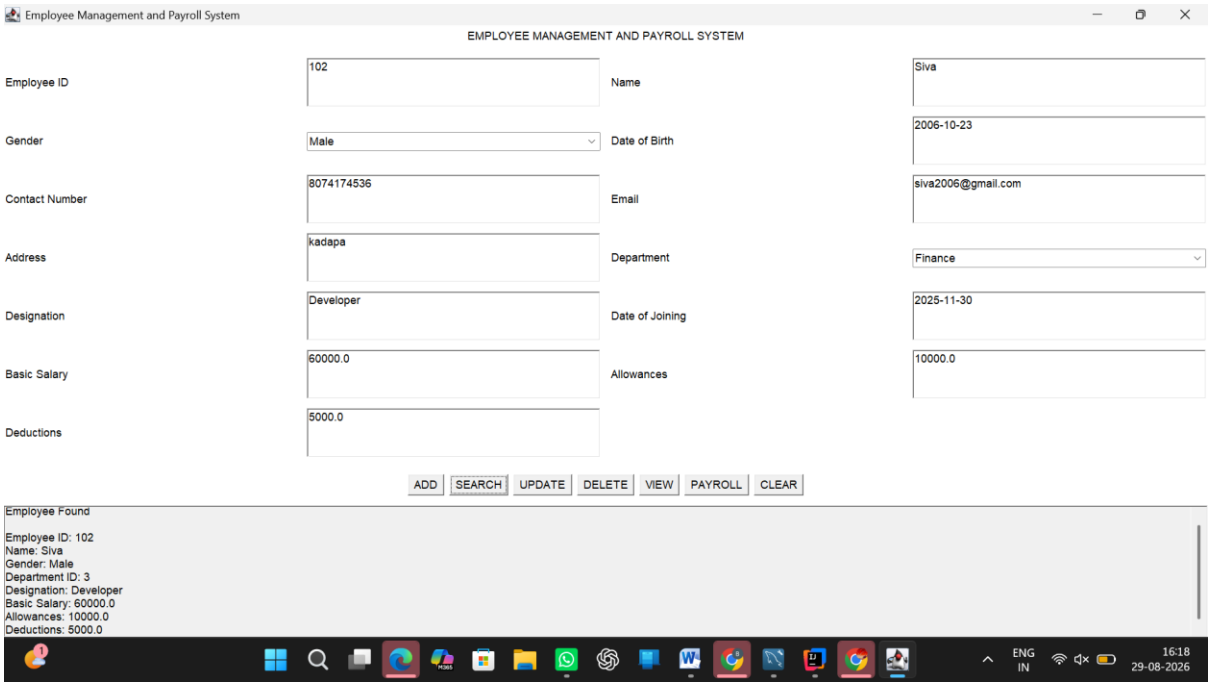
PAYROLL:



VIEW:


Employee Management and Payroll System
EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM
Employee ID
101
Name
Bala
Gender
Male
Date of Birth
2006-11-24
Contact Number
8074171936
Email
balasiva2006@gmail.com
Address
kadapa
Department
IT
Designation
Developer
Date of Joining
2025-11-30
Basic Salary
50000
Allowances
20000
Deductions
4000
ADD
SEARCH
UPDATE
DELETE
VIEW
PAYROLL
CLEAR
ID: 101
Name: Bala
Department: IT
Designation: Developer
Basic Salary: 50000.0
Allowances: 20000.0
Deductions: 4000.0
Status: Active
ENG
IN
16:16
29-08-2026

SEARCH:


Employee Management and Payroll System
EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM
Employee ID
102
Name
Siva
Gender
Male
Date of Birth
2006-10-23
Contact Number
8074174536
Email
siva2006@gmail.com
Address
kadapa
Department
Finance
Designation
Developer
Date of Joining
2025-11-30
Basic Salary
60000.0
Allowances
10000.0
Deductions
5000.0
ADD
SEARCH
UPDATE
DELETE
VIEW
PAYROLL
CLEAR
Employee Found
Employee ID: 102
Name: Siva
Gender: Male
Department ID: 3
Designation: Developer
Basic Salary: 60000.0
Allowances: 10000.0
Deductions: 5000.0
ENG
IN
16:18
29-08-2026